

➤ Event Sources

A source is an object that generates an event. Sources may generate more than one type of event. A source must register listeners in order for the listeners to receive notifications about a specific type of event. Here is the general form:

1. ***public void addTypeListener (TypeListener el)***

Here, Type is the name of the event, and el is a reference to the event listener.

When an event occurs, all registered listeners are notified and receive a copy of the event object. This is known as **multicasting the event**.

Some sources may allow only one listener to register. The general form of such a method is this:

2. ***public void addTypeListener(TypeListener el) throws java.util.TooManyListenersException***

This is known as **unicasting** the event.

A source must also provide a method that allows a listener to unregister. The general form of such a method is this:

3. ***public void removeTypeListener(TypeListener el)***

For example, to remove a keyboard listener, you would call `removeKeyListener()`.

➤ Event Listeners

A listener is an object that is notified when an event occurs. The listener must supply the event handlers in form of registration and implementation of corresponding method. Methods are defined in a set of interfaces.

An event handler must return quickly. It must perform specific actions in response to events and then return control to the run-time system. Failure to do this can cause your program to appear sluggish or even non-responsive.

➤ Event Classes

At the root of the Java event class hierarchy is `EventObject`, which is in `java.util`. It is the superclass for all events. It's one constructor is shown here:

- `EventObject(Object src)`

Here, `src` is the object that generates this event. `EventObject` defines two methods: `getSource()` and `toString()`. The `getSource()` method returns the source of the event. Its general form is shown here:

- `Object getSource()`

As expected, `toString()` returns the string equivalent of the event.

`getID()` method can be used to determine the type of the event. The signature of this method is shown here:

- `int getID()`

➤ The ActionEvent Class

An `ActionEvent` is generated when a button is pressed, a list item is double-clicked, or a menu item is selected. The `ActionEvent` class defines four integer constants that can be used to identify any modifiers associated with an action event: `ALT_MASK`, `CTRL_MASK`, `META_MASK`, and `SHIFT_MASK`. In addition, there is an integer constant, `ACTION_PERFORMED`, which can be used to identify action events.

`ActionEvent` has these three constructors:

- `ActionEvent(Object src, int type, String cmd)`
- `ActionEvent(Object src, int type, String cmd, int modifiers)`
- `ActionEvent(Object src, int type, String cmd, long when, int modifiers)`

Here, `src` is a reference to the object that generated this event. The type of the event is specified by `type`, and its command string is `cmd`. The argument `modifiers` indicates which modifier keys (alt, ctrl, meta, and/or shift) were pressed when the event was generated. The `when` parameter specifies when the event occurred. You can obtain the command name for the invoking `ActionEvent` object by using the `getActionCommand( )` method, shown here:

- `String getActionCommand( )`

For example, when a button is pressed, an action event is generated that has a command name equal to the label on that button.

The `getModifiers( )` method returns a value that indicates which modifier keys (`ALT`, `CTRL`, `META`, and/or `SHIFT`) were pressed when the event was generated. Its form is shown here:

- `int getModifiers( )`
- **The `ComponentEvent` Class**

A `ComponentEvent` is generated when the size, position, or visibility of a component is changed. There are four types of component events. The `ComponentEvent` class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

<code>COMPONENT_HIDDEN</code>	The component was hidden.
<code>COMPONENT_MOVED</code>	The component was moved.
<code>COMPONENT_RESIZED</code>	The component was resized.
<code>COMPONENT_SHOWN</code>	The component became visible.

`ComponentEvent` has this constructor:

- `ComponentEvent(Component src, int type)`

➤ **The `ContainerEvent` Class**

A ContainerEvent is generated when a component is added to or removed from a container. There are two types of container events. The ContainerEvent class defines int constants that can be used to identify them: COMPONENT_ADDED and COMPONENT_REMOVED. They indicate that a component has been added to or removed from the container.

ContainerEvent is a subclass of ComponentEvent and has this constructor:

- ContainerEvent(Component src, int type, Component comp)

➤ **The InputEvent Class**

The abstract class InputEvent is a subclass of ComponentEvent and is the superclass for component input events. Its subclasses are KeyEvent and MouseEvent.

InputEvent defines several integer constants that represent any modifiers, such as the control key being pressed, that might be associated with the event.

It is possible to obtain a value that contains all of the original modifier flags by calling the getModifiers() method. It is shown here:

int getModifiers()

➤ **The ItemEvent Class**

An ItemEvent is generated when a check box or a list item is clicked or when a checkable menu item is selected or deselected.

In addition, ItemEvent defines the integer constant, ITEM_STATE_CHANGED, that signifies a change of state.

ItemEvent has this constructor:

- ItemEvent(ItemSelectable src, int type, Object entry, int state)

The getItem() method can be used to obtain a reference to the item that changed. Its signature is shown here:

- Object getItem()

The getItemSelectable() method can be used to obtain a reference to the ItemSelectable object that generated an event. Its general form is shown here:

- ItemSelectable getItemSelectable()

➤ **The KeyEvent Class**

A KeyEvent is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants: KEY_PRESSED, KEY_RELEASED, and KEY_TYPED. There are many other integer constants that are defined by KeyEvent. For example, VK_0 through VK_9 and VK_A through VK_Z define the ASCII equivalents of the numbers and letters.

VK_ALT	VK_DOWN	VK_LEFT	VK_RIGHT
VK_CANCEL	VK_ENTER	VK_PAGE_DOWN	VK_SHIFT
VK_CONTROL	VK_ESCAPE	VK_PAGE_UP	VK_UP

KeyEvent is a subclass of InputEvent. Here is one of its constructors:

- KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)

The KeyEvent class defines several methods, but probably the most commonly used ones are getKeyChar(), which returns the character that was entered, and getKeyCode(), which returns the key code. Their general forms are shown here:

- char getKeyChar()
- int getKeyCode()

➤ The MouseEvent Class

There are eight types of mouse events. The MouseEvent class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

Here is one of its constructors:

- MouseEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup)

The modifiers argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in x and y. The click count is passed in clicks. The triggersPopup flag indicates if this event causes a pop-up menu to appear on this platform. Two commonly used methods in this class are getX() and getY(). These return the X and Y coordinates of the mouse within the component when the event occurred. Their forms are shown here:

- int getX()
- int getY()

The `translatePoint( )` method changes the location of the event. Its form is shown here:

- `void translatePoint(int x, int y)`

Here, the arguments `x` and `y` are added to the coordinates of the event. The `getClickCount( )` method obtains the number of mouse clicks for this event. Its signature is shown here:

- `int getClickCount( )`

➤ **The WindowEvent Class**

There are ten types of window events. The `WindowEvent` class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

<code>WINDOW_ACTIVATED</code>	The window was activated.
<code>WINDOW_CLOSED</code>	The window has been closed.
<code>WINDOW_CLOSING</code>	The user requested that the window be closed.
<code>WINDOW_DEACTIVATED</code>	The window was deactivated.
<code>WINDOW_DEICONIFIED</code>	The window was deiconified.
<code>WINDOW_GAINED_FOCUS</code>	The window gained input focus.
<code>WINDOW_ICONIFIED</code>	The window was iconified.
<code>WINDOW_LOST_FOCUS</code>	The window lost input focus.
<code>WINDOW_OPENED</code>	The window was opened.
<code>WINDOW_STATE_CHANGED</code>	The state of the window changed.

`WindowEvent` is a subclass of `ComponentEvent`. It defines several constructors. The first is

- `WindowEvent(Window src, int type)`

Here, `src` is a reference to the object that generated this event. The type of the event is `type`.

The next three constructors offer more detailed control:

- `WindowEvent(Window src, int type, Window other)`
- `WindowEvent(Window src, int type, int fromState, int toState)`
- `WindowEvent(Window src, int type, Window other, int fromState, int toState)`

Here, `other` specifies the opposite window when a focus or activation event occurs. The `fromState` specifies the prior state of the window, and `toState` specifies the new state that the window will have when a window state change occurs.

Event Listener Interfaces

➤ The ActionListener Interface

This interface defines the `actionPerformed( )` method that is invoked when an action event occurs. Its general form is shown here:

- `void actionPerformed(ActionEvent ae)`

➤ The AdjustmentListener Interface

This interface defines the `adjustmentValueChanged( )` method that is invoked when an adjustment event occurs. Its general form is shown here:

- `void adjustmentValueChanged(AdjustmentEvent ae)`

➤ The ComponentListener Interface

This interface defines four methods that are invoked when a component is resized, moved, shown, or hidden. Their general forms are shown here:

- `void componentResized(ComponentEvent ce)`
- `void componentMoved(ComponentEvent ce)`
- `void componentShown(ComponentEvent ce)`
- `void componentHidden(ComponentEvent ce)`

The ContainerListener Interface This interface contains two methods. When a component is added to a container, `componentAdded( )` is invoked. When a component is removed from a container, `componentRemoved( )` is invoked. Their general forms are shown here:

- `void componentAdded(ContainerEvent ce)`
- `void componentRemoved(ContainerEvent ce)`

➤ The ItemListener Interface

This interface defines the `itemStateChanged( )` method that is invoked when the state of an item changes. Its general form is shown here:

- `void itemStateChanged(ItemEvent ie)`

➤ **The KeyListener Interface**

This interface defines three methods. The keyPressed() and keyReleased() methods are invoked when a key is pressed and released, respectively. The keyTyped() method is invoked when a character has been entered.

The general forms of these methods are shown here:

- void keyPressed(KeyEvent ke)
- void keyReleased(KeyEvent ke)
- void keyTyped(KeyEvent ke)

➤ **The MouseListener Interface**

This interface defines five methods. The general forms of these methods are shown here:

- void mouseClicked(MouseEvent me)
- void mouseEntered(MouseEvent me)
- void mouseExited(MouseEvent me)
- void mousePressed(MouseEvent me)
- void mouseReleased(MouseEvent me)

➤ **The MouseMotionListener Interface**

This interface defines two methods. Their general forms are shown here:

- void mouseDragged(MouseEvent me)
- void mouseMoved(MouseEvent me)

➤ **The WindowListener Interface**

This interface defines seven methods. The general forms of these methods are

- void windowActivated(WindowEvent we)
- void windowClosed(WindowEvent we)
- void windowClosing(WindowEvent we)
- void windowDeactivated(WindowEvent we)
- void windowDeiconified(WindowEvent we)
- void windowIconified(WindowEvent we)

- void windowOpened(WindowEvent we)

Inner Classes

An inner class is a class defined within another class, or even within an expression. Inner classes can be used to simplify the code when using event adapter classes.

```
// This program does NOT use an inner class.
import java.awt.*;
import java.awt.event.*;
```

```
public class MousePressedDemo extends Frame {
    String msg = "";
```

```
    public MousePressedDemo() {
        addMouseListener(new MyMouseAdapter(this));
        addWindowListener(new MyWindowAdapter());
    }

    public void paint(Graphics g) {
        g.drawString(msg, 20, 100);
    }

    public static void main(String[] args) {
        MousePressedDemo appwin = new MousePressedDemo();

        appwin.setSize(new Dimension(200, 150));
        appwin.setTitle("MousePressedDemo");
        appwin.setVisible(true);
    }
}

class MyMouseAdapter extends MouseAdapter {
    MousePressedDemo mousePressedDemo;

    public MyMouseAdapter(MousePressedDemo mousePressedDemo) {
        this.mousePressedDemo = mousePressedDemo;
    }

    // Handle a mouse pressed.
    public void mousePressed(MouseEvent me) {
        mousePressedDemo.msg = "Mouse Pressed.";
        mousePressedDemo.repaint();
    }
}

// When the close box in the frame is clicked,
// close the window and exit the program.
class MyWindowAdapter extends WindowAdapter {
    public void windowClosing(WindowEvent we) {
        System.exit(0);
    }
}
```


